

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: FPGA Based System Design (01CT0718)	Aim: To design and implement FSM-based controllers for sequential digital systems.	
Experiment No: 9	Date:	Enrolment No: 92301733013

Exp_

Aim: To design and implement FSM-based controllers for sequential digital systems.

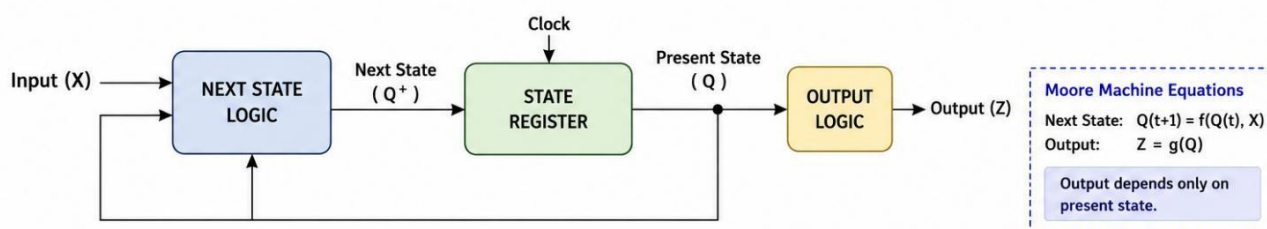
Software required: Xilinx Vivado

Hardware required: Nexys4 DDR FPGA Development Board

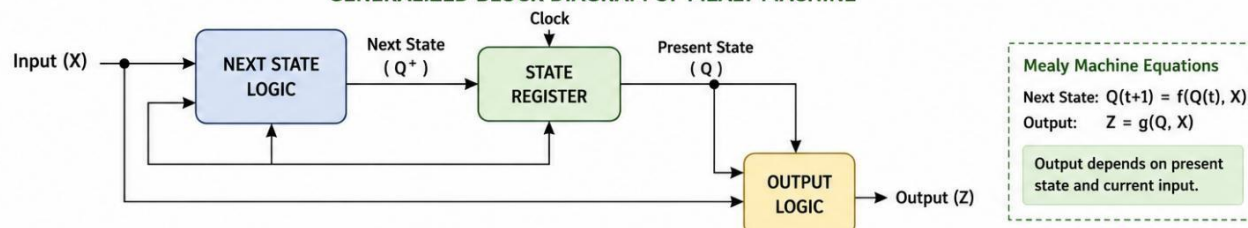
Theory:

Block diagram:

GENERALIZED BLOCK DIAGRAM OF MOORE MACHINE



GENERALIZED BLOCK DIAGRAM OF MEALY MACHINE

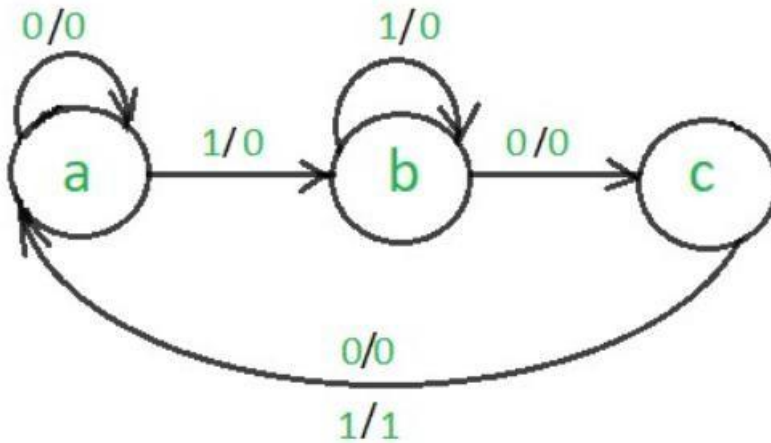


MOORE vs MEALY MACHINE

Feature	Moore Machine	Mealy Machine
Output depends on	Present State only	Present State and Input
Input connected to Output Logic	No	Yes
Output changes	After state changes (on clock edge)	Immediately when input changes
Output stability	More stable	Can have glitches
Response speed	Slower	Faster

LEGEND	
	Next State Logic
	State Register
	Output Logic
X	= Input
Q	= Present State
Q ⁺	= Next State
Z	= Output

Mealy machine for sequence 101
State Diagram

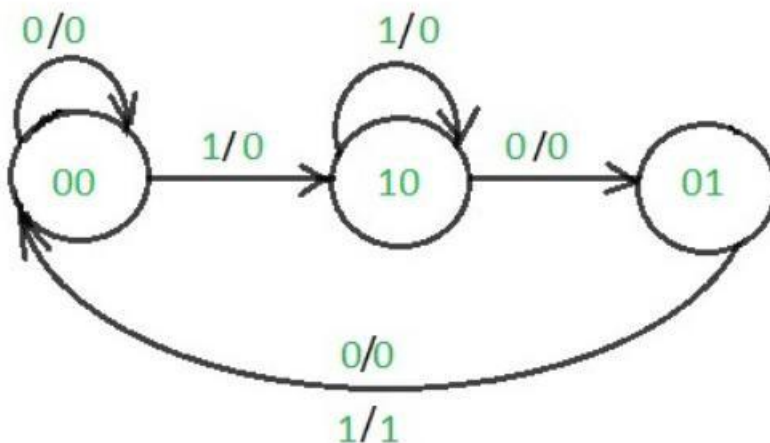


State assignment

Previous States	States	Next States
a,c	a	a,b
b,a	b	b,c
b	c	a

x \ y	0	1
0	a	b
1	c	

State Diagram after State Diagram





State table

Present States		i/p	Next States		Flip Flop Excitations		O/P
X	Y		X'	Y'	Dx	Dy	
0	0	0	0	0	0	0	0
0	0	1	1	0	1	0	0
0	1	0	0	0	0	0	0
0	1	1	0	0	0	0	1
1	0	0	0	1	0	1	0
1	0	1	1	0	1	0	0
1	1	0	X	X	X	X	X
1	1	1	X	X	X	X	X

Boolean Equations formation

I	XY			
	00	01	11	10
0	0	0	X	0
1	1	0	X	1

$$Dx = \overline{Y}.I$$

I	XY			
	00	01	11	10
0	0	0	X	1
1	0	0	X	0

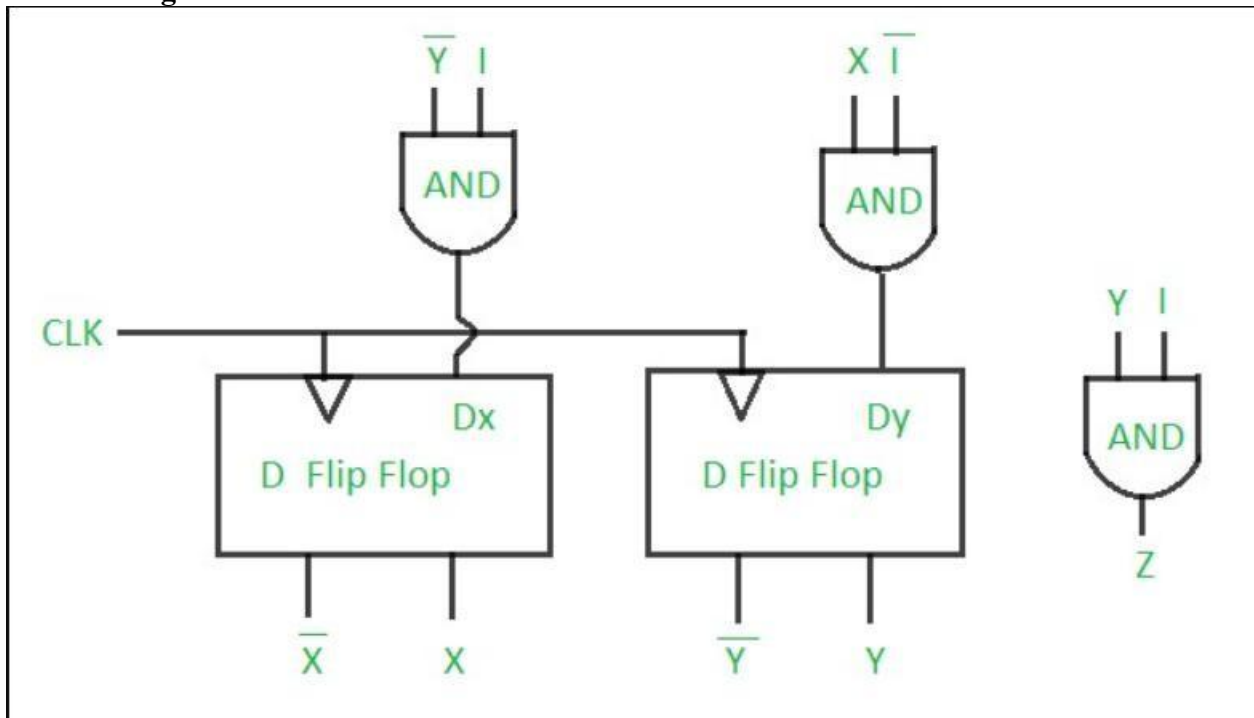
$$Dy = X.\overline{I}$$

I	XY			
	00	01	11	10
0	0	0	X	0
1	0	1	X	0

$$Z = Y.I$$

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: FPGA Based System Design (01CT0718)	Aim: To design and implement FSM-based controllers for sequential digital systems.	
Experiment No: 9	Date:	Enrolment No: 92301733013

Circuit Diagram



Verilog Code:

```

`timescale 1ns / 1ps

module fsm_mealy (
    input wire CLK100MHZ,
    input wire BTNC,
    input wire SW0,

    output wire LED0,
    output wire LED1
);

// State definitions
localparam S0 = 2'b00;
localparam S1 = 2'b01;
localparam S2 = 2'b10;

reg [1:0] state;
reg [1:0] next_state;

reg [25:0] counter;

wire sample_tick;

```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: FPGA Based System Design (01CT0718)	Aim: To design and implement FSM-based controllers for sequential digital systems.	
Experiment No: 9	Date:	Enrolment No: 92301733013

```
assign sample_tick = (counter == 26'd49_999_999);
```

```
always @(posedge CLK100MHZ or posedge BTNC) begin
    if (BTNC)
        counter <= 26'd0;
```

```
        elseif(sample_tick
        ) counter <= 26'd0;
    else
        counter <= counter + 1'b1;
end
```

```
// Next-state logic
always @(*) begin
```

```
    case (state)
```

```
        // No matching bits
        S0: begin
            if (SW0 == 1'b1)
                next_state = S1;
            else
                next_state = S0;
        end
```

```
        // Received "1"
        S1: begin
            if (SW0 == 1'b0)
                next_state = S2;
            else
                next_state = S1;
        end
```

```
        // Received "10"
        S2: begin
            if (SW0 == 1'b1)
                next_state = S1;
            else
                next_state = S0;
        end
```

```
        default:
            next_state = S0;
```

```
    endcase
```

```
end
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: FPGA Based System Design (01CT0718)	Aim: To design and implement FSM-based controllers for sequential digital systems.	
Experiment No: 9	Date:	Enrolment No: 92301733013

```
// State register + Mealy detection output
reg detection;
```

```
always @(posedge CLK100MHZ or posedge BTNC) begin
```

```
    if (BTNC) begin
        state    <= S0;
        detection <= 1'b0;
    end
```

```
    else if (sample_tick) begin
```

```
        if ((state == S2) && (SW0 == 1'b1))
            detection <= 1'b1;
        else
            detection <= 1'b0;
```

```
        state <= next_state;
```

```
    end
```

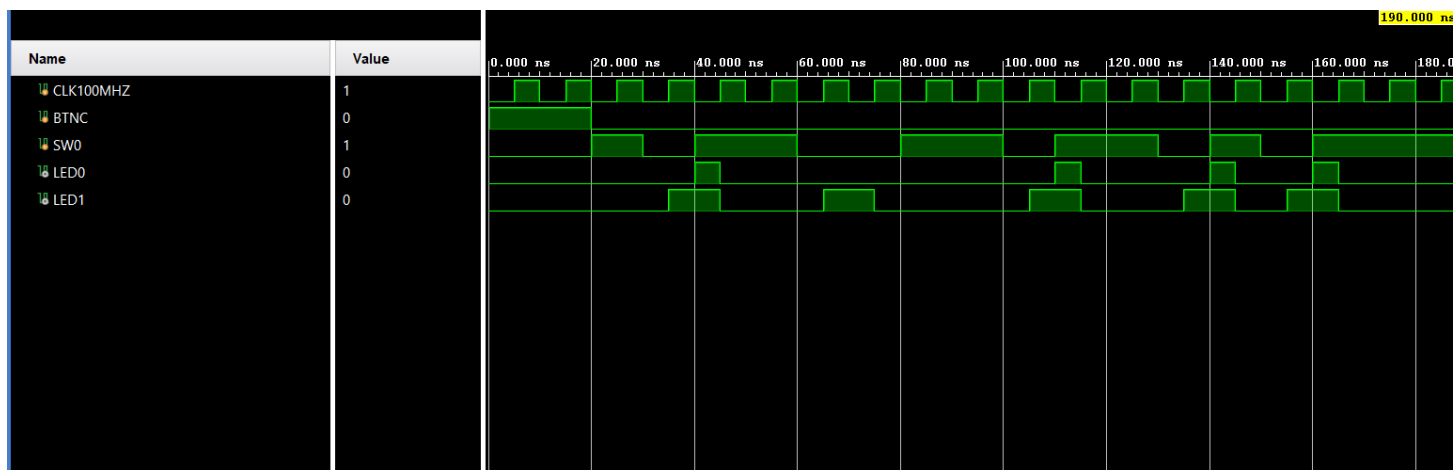
```
end
```

```
// Outputs
```

```
// LED0 = 101 detected
assign LED0 = detection;
```

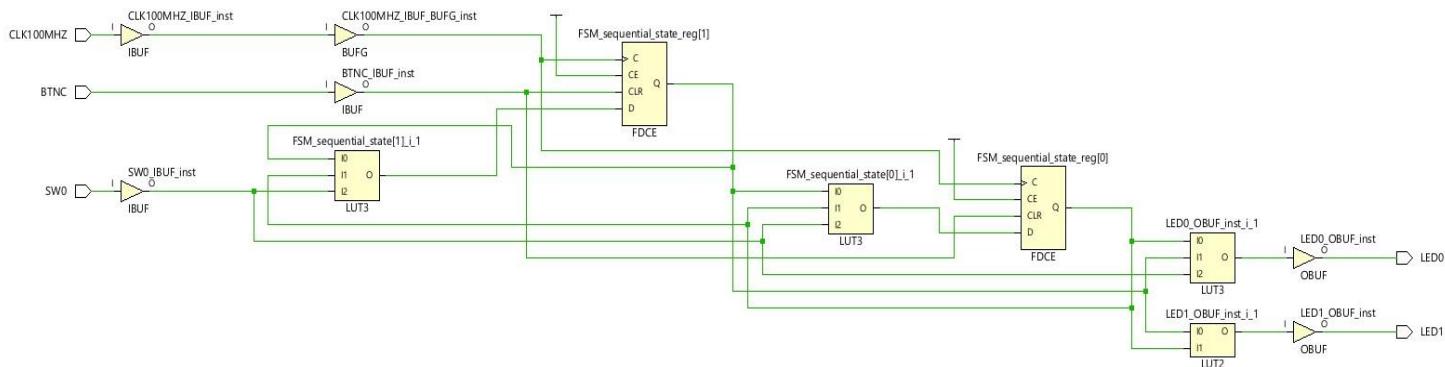
```
// LED1 = FSM currently in S2 ("10")
assign LED1 = (state == S2);
```

```
endmodule
```

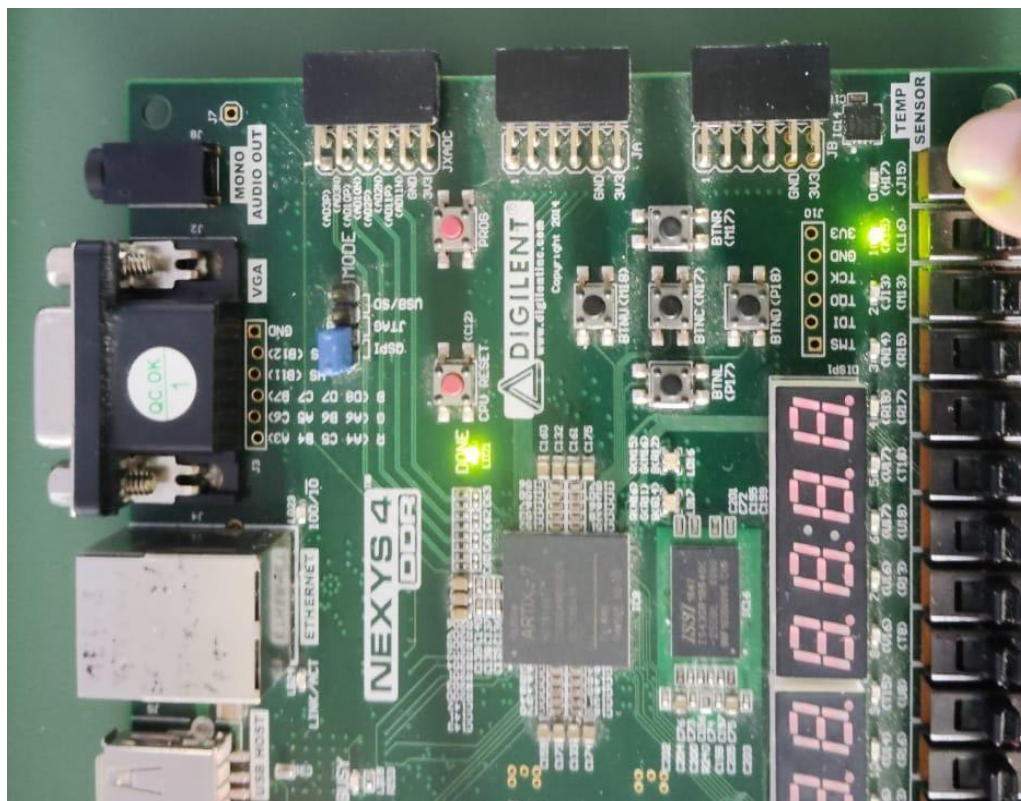


 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: FPGA Based System Design (01CT0718)	Aim: To design and implement FSM-based controllers for sequential digital systems.	
Experiment No: 9	Date:	Enrolment No: 92301733013

Schematic:



FPGA Implementation



 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: FPGA Based System Design (01CT0718)	Aim: To design and implement FSM-based controllers for sequential digital systems.	
Experiment No: 9	Date:	Enrolment No: 92301733013

Conclusion:

The Mealy FSM for detecting the 101 sequence was successfully designed and implemented in Verilog using Xilinx Vivado. The states, transitions, Boolean logic, and overall circuit behavior were verified through simulation. The results confirmed that the FSM correctly detects the required sequence and operates as expected on the FPGA.

Post Lab Exercise:

1. Why is a Mealy FSM generally faster than a Moore FSM?

A Mealy FSM can respond faster because its output depends on both the current state and the input. Therefore, the output can change as soon as the input changes. In contrast, a Moore FSM generates its output based only on the current state, so the output changes after the state transition at the clock edge, which can result in a one-clock-cycle delay.

2. How does allowing overlapping sequences affect the number of FSM states?

Allowing overlapping sequences can reduce the number of states required in an FSM because previously matched bits can be reused instead of resetting to the initial state after detecting a sequence. This makes the overall FSM design simpler and more efficient.

3. Why is state minimization important in FSM design?

- Reduces the number of flip-flops and logic gates.
- Lowers hardware cost and power consumption.
- Improves speed and simplifies circuit design.

4. What is the impact of an incorrect state transition in an FSM?

- Produce incorrect outputs.
- Detect wrong sequences or miss valid ones.
- Cause the controller to enter an invalid or unexpected state.
- Reduce system reliability and may lead to system failure.

5. Why is state encoding important in digital system implementation?

- Reduces logic complexity.
- Improves circuit speed.
- Minimizes power consumption.
- Optimizes hardware resource usage.

6. How does one-hot encoding improve FPGA-based FSM performance?

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: FPGA Based System Design (01CT0718)	Aim: To design and implement FSM-based controllers for sequential digital systems.	
Experiment No: 9	Date:	Enrolment No: 92301733013

- Simplifies combinational logic.
- Reduces propagation delay.
- Increases operating speed on FPGAs, which have abundant flip-flops.

7. How does an FSM improve the reliability of a digital controller?

- Providing predictable and controlled state transitions.
- Preventing invalid operations.
- Ensuring the system follows a predefined sequence.
- Making debugging and verification easier.

8. Why are reset states essential in FSM design?

- Initializes the FSM to a known starting state during power-up or reset.
- Prevents unpredictable behavior.
- Allows the system to recover from errors.
- Ensures correct operation from the beginning.